Algorithmic Bias Audit Report

Target File: `./test/assign_field_support_.py`

Audit Model: FATE Auditor Model

Analysis Type: Static Code FATE Analysis

Status: 4 Flags Detected

Executive Summary

This report details instances of algorithmic bias identified within the source code during automated static analysis. The evaluation surfaced exclusionary logic based on protected attributes and/or proxies, which may propagate differential treatment through the Software Development Life Cycle (SDLC).

Detailed Findings

Finding 1: Proxy Bias

Association: NEGATIVE

Non-Protected Attribute: can_work_without_accommodations

Biased Code Snippet:

```
if engineer["can_work_without_accommodations"] == True:
```

Issue Description: The function uses 'can_work_without_accommodations' as a proxy for the protected attribute 'Disability', excluding individuals who may require accommodations from being assigned to field support.

Impact Reasoning: Relies on a measurable attribute ('can_work_without_accommodations') as a stand-in for a trait that correlates with a protected group, unfairly impacting groups where that attribute does not accurately indicate the intended outcome.

Finding 2: Demographic Bias

Association: NEGATIVE

Non-Protected Attribute: can_work_without_accommodations

Biased Code Snippet:

```
if engineer["can_work_without_accommodations"] == True:
```

Issue Description: The function uses the attribute 'can_work_without_accommodations' to exclude a group of individuals from a work assignment, leading to differential treatment based on a proxy for disability.

Impact Reasoning: Uses a non-protected attribute ('can_work_without_accommodations') to result in differential treatment of individuals based on group membership or proxy grouping.

Finding 3: Threshold/Cutoff Bias

Association: NEGATIVE

Non-Protected Attribute: can_work_without_accommodations

Biased Code Snippet:

```
if engineer["can_work_without_accommodations"] == True:
```

Issue Description: The code uses a rigid boolean check on 'can_work_without_accommodations', creating a strict cutoff that disqualifies anyone needing any level of accommodation without nuance.

Impact Reasoning: Uses a fixed non-protected attribute ('can_work_without_accommodations == True') with a threshold to determine eligibility or outcomes, where the rigid cutoff may produce biased treatment for individuals near the boundary.

Finding 4: Stereotype Bias

Association: NEGATIVE

Non-Protected Attribute: can_work_without_accommodations

Biased Code Snippet:

```
if engineer["can_work_without_accommodations"] == True:
```

Issue Description: The code implies that needing accommodations makes an engineer unsuitable for 'field_support', reinforcing a negative stereotype about the capabilities of individuals with disabilities.

Impact Reasoning: Associates ('unsuitability for field support') with a non-protected attribute, reinforcing a negative stereotype in the program logic.